

PolyCare — Session Worklog

2026-09-28 · Restarted session, working from LovekeshAnand/polymath upstream **c9a805a** (“milestone-zero/one”) · Records only the work performed in this session

1. Context

This session was explicitly restarted: the local repository was reset to the colleague's (LovekeshAnand) latest upstream **main** branch, and an earlier, now-abandoned session's work was set aside as a stash/backup branch. Everything below was performed against that clean, synced starting point — nothing from the earlier session was reused or merged in.

Before any code was written, Milestones 0 and 1 were audited from the actual current repository state (code, tests, STATUS.md, WORKLOG.md) rather than assumed. The audit found the repository much further along than a first glance suggested: real native Qdrant Edge (Rust/UniFFI, arm64), a real multilingual embedder (ONNX, hand-ported SentencePiece tokenizer), and a real knowledge base built from official Government of India NHM/MoHFW ASHA training material.

2. Independent verification of M0 and M1 (not assumed)

This session ran on a second, previously bare machine, with a second test phone (**Realme RMX2151, Android 12**) different from the colleague's Xiaomi. That made it possible to independently re-verify — not just re-read — every M0/M1 claim that a real device could confirm.

Check	Result on this machine / phone
Full unit test suite (fresh checkout)	All tests pass, every module
Qdrant Edge on-device instrumented tests	3/3 pass on the Realme — matches the Xiaomi's 3/3
Embedder JVM parity test	Passes against the colleague's committed reference fixture unchanged
Embedder on-device self-check	PASS — minCos 0.998317, 25/25 nearest-neighbour agreement (matches Xiaomi's 0.9983)
Knowledge base install + real search on phone	Embed ~20–22 ms, search ~28–45 ms

Conclusion: M0 is genuinely 5/10 complete and M1 genuinely 4/5, independently confirmed on a second phone, not just restated from documentation. Nothing in the remaining M0/M1 items (llama.cpp, LoRA adapters, whisper.cpp, ML Kit OCR, Qdrant Cloud, Memory Inspector screen) blocks Milestone 2 — Memory Inspector's own backend (`KnowledgeRepository.browse().stats()`) already exists, only its screen is missing, and it was left untouched since nothing in M2 depends on it.

3. New-machine toolchain setup

This machine had no Android SDK, no Rust, and no native build tools. Everything below was installed and debugged from scratch to make the existing native Qdrant Edge build (`native/build-qdrant-edge.sh`) actually run here, matching the colleague's already-working setup rather than skipping or faking it.

- Android SDK command-line tools, platform-tools, platform 35, build-tools 35.0.0, NDK 27.1.12297006.
- Gradle needed a JDK 17 toolchain the machine didn't have (only JDK 21 was installed) — fixed by adding the `org.gradle.toolchains.foojay-resolver-convention` plugin so Gradle provisions its own JDK 17 instead of requiring a pre-installed one.

- Rust via rustup, the aarch64-linux-android target, cargo-ndk, and MinGW-w64 (WinLibs).

The space-in-username problem

This account's Windows profile path is `C:\Users\Dua Saeed\...` — the embedded space broke MinGW's linker in two separate, non-obvious ways, each requiring its own fix:

- Rustup's default host toolchain is MSVC, not GNU — the build script needs the GNU host (so MinGW's linker is used at all). Fixed by switching to `rustup default stable-x86_64-pc-windows-gnu`.
- Even with the GNU toolchain and `RUSTUP_HOME/CARGO_HOME` relocated to a space-free path, linking still failed with `"C:/Users/Dua: file not recognized."` Traced to MinGW-w64 itself being installed under the space-containing WinGet package directory, which GCC resolves its own internal tool paths from. Fixed by copying `mingw64/` to a space-free `C:\mingw64` and pointing `PATH` there instead.

With both fixed, the native build (`cargo ndk ... build -p qdrant-edge-ffi`) completed cleanly in 33m 42s, producing `libqdrant_edge_ffi.so` (36 MB, arm64-v8a). The already-committed Kotlin UniFFI bindings needed no regeneration (same pinned upstream commit).

4. Milestone 2: Ask and Triage

The repository's own `MILESTONES.md` scopes M2 to the offline Ask/Triage assistant, explicitly with **zero cloud dependency**. A separate request in this same conversation, about location-aware data synchronization, maps to M6/M7/M8 in the repository's own numbering and was intentionally **not** built this session — no cloud gateway or op-log exists yet to build real sync against, and conflating it with M2 would have made the milestone documentation internally inconsistent.

Ask

Wraps the existing, real `KnowledgeRepository` (the same hybrid search `Search` already uses). Shows the best-matching passage with its source, page, and a confidence badge. Confidence is a simple, explainable term-overlap measure (fraction of the question's words found in the answer) rather than the RRF rank score, which isn't a bounded similarity measure. Below the existing `PolyCareConfig.Routing.minSkillScore` threshold (reused, not duplicated), the UI shows a low-confidence warning and logs a gap.

There is no on-device LLM yet. Rather than fabricate a generated answer, Ask shows the retrieved passage directly and labels it as such — which is exactly the Resource Governor's own documented `RECALL` rung ("no LLM: shows the best protocol passages"), so this is the system's own designed degradation behaviour, not an improvised workaround.

Triage

A rule table for newborn / child / postpartum danger signs, citing the same official ASHA module sources already indexed for Search and Ask, rather than inventing separate citations. The referral decision always comes from the rule table (never a model); the explanation text is a clearly-labelled template until the on-device LLM lands. Covered by 5 unit tests exercising every decision branch.

Gaps

An in-memory, HLC-timestamped log of unanswered or low-confidence questions, populated automatically by Ask. Its own documentation notes that this moves behind the op-log in M4 and syncs to the cloud in M7 — designed not to block that later, not built ahead of it.

5. On-device verification of the new work

This phone (like the colleague's) blocks adb input taps, so two debug launch extras were added, matching the existing search_query pattern, to drive the new screens from the PC without tapping:

Command	Result
--es ask_query "baby has fast breathing"	Confidence 1.00, source "asha-induction", correctly not flagged low-confidence, event logged
--ez open_triage true	Screen renders; MainActivity confirmed in foreground; no FATAL/AndroidRuntime in logcat

Full unit test suite re-run after all changes: **all tests pass**, every module, including the new TriageEngineTest (5/5).

6. Deliberately not built this session

- Voice input, skill routing/blending, LLM-generated explanations, speculative decoding gauge — all need llama.cpp, which the colleague has correctly not yet integrated (M0).
- Dedicated “medicine helper” card UI — the content is indexed and searchable via Ask/Search already; a separate card screen wasn't built, to avoid scope creep beyond what M2 actually requires.
- Memory Inspector screen (M1) — confirmed not to block M2; left for later as originally scoped.
- Data synchronization / location-awareness — a separate request mapping to M6/M7/M8, needing a cloud gateway and op-log that don't exist yet; intentionally out of scope for this session pending direction on how to reconcile it with the repository's own milestone numbering.

7. Files changed this session

New: android/app/src/main/kotlin/org/polycare/app/ask/{AskScreen,AskViewModel}.kt ·
android/app/src/main/kotlin/org/polycare/app/triage/{TriageScreen,TriageViewModel}.kt ·
android/app/src/main/kotlin/org/polycare/app/knowledge/{TriageEngine,GapsRepository}.kt ·
android/app/src/test/kotlin/org/polycare/app/knowledge/TriageEngineTest.kt
Edited: android/app/src/main/kotlin/org/polycare/app/{MainActivity.kt, di/AppModule.kt, home/HomeScreen.kt, ui/PolyCareRoot.kt} ·
android/core-common/src/main/kotlin/org/polycare/common/EventLog.kt · android/settings.gradle.kts (Foojay resolver) ·
MILESTONES.md · STATUS.md · WORKLOG.md · README.md (status badge, roadmap table)
Generated locally, not committed to git: android/qdrant-edge/src/main/jniLibs/arm64-v8a/libqdrant_edge_ffi.so ·
tools/models/multilingual-e5-small/* (gitignored, as designed)

This document records only what was verified or built in this restarted session. It does not restate work from earlier sessions; see WORKLOG.md in the repository for the full project history and STATUS.md for the always-current milestone dashboard.